

Projet "Audit & Hardening Lab" — Guide de réalisation complet

Candidature MS-SIS ESIEA

Vue d'ensemble

Ce projet consiste à construire un mini-lab de cybersécurité combinant deux volets complémentaires : l'audit automatisé d'équipements réseau et le durcissement d'un serveur Linux exposé. L'objectif est de démontrer une démarche complète d'expert SSI : état initial vulnérable, collecte de preuves, analyse, remédiation automatisée, puis mesure de l'amélioration obtenue.

Le projet est particulièrement pertinent pour une candidature au Mastère Spécialisé Sécurité de l'Information et des Systèmes (MS-SIS) de l'ESIEA, dont le positionnement couvre la sécurité des infrastructures, les systèmes, l'analyse technique et une culture proche des référentiels français de cybersécurité comme l'ANSSI.

Objectifs du projet

- Construire un environnement reproductible avec Docker et Ansible.
- Simuler un ou plusieurs équipements réseau présentant des écarts de sécurité.

- Déployer un serveur Linux intentionnellement mal configuré pour servir de cible de durcissement.
- Réaliser un audit automatisé de l'existant avec Lynis et des playbooks Ansible.
- Appliquer un hardening Linux basé sur le référentiel ANSSI-BP-028.
- Mesurer l'écart avant/après avec Lynis.
- Générer un rapport lisible par un recruteur, un jury ou un tuteur technique.

Architecture cible

Composants

Composant	Rôle
Contrôleur Ansible	Lance les playbooks d'audit et de hardening
Serveur Linux cible	Cible Ubuntu Server mal configurée
FakeNOS / équipement simulé	Équipement réseau avec vulnérabilités
Dossier reports/	Stocke les résultats JSON, CSV, HTML
Pipeline CI (optionnel)	Rejoue les vérifications à chaque push

Flux logique

```
Déploiement du lab
↓
Audit initial (baseline)
↓
Analyse et scoring
↓
Hardening automatisé (Ansible)
↓
Audit de validation
```

↓
Rapport comparatif avant/après

Choix technologiques

Outil	Rôle dans le projet
Docker Compose	Déployer rapidement un lab reproductible
Ansible	Collecte, durcissement, conformité
Python	Parser les résultats, calculer les écarts
Lynis	Mesurer le niveau de hardening avant/après
auditd	Tracer les événements sensibles
nftables / iptables	Réduire la surface d'exposition
SSH + clés	Supprimer les accès faibles
Markdown / Jinja2	Produire un rapport lisible

Arborescence recommandée du dépôt

```
audit-hardening-lab/  
├─ docker-compose.yml  
├─ inventory/  
│   ├── hosts.yml  
│   └─ group_vars/  
├─ playbooks/  
│   ├── deploy_lab.yml  
│   ├── audit_linux.yml  
│   ├── audit_network.yml  
│   ├── harden_linux.yml  
│   └─ validate.yml  
├─ roles/  
│   ├── linux_baseline/  
│   ├── linux_hardening/  
│   │   ├── tasks/  
│   │   │   ├── ssh.yml  
│   │   └─ firewall.yml
```

```
| | | | |— services.yml
| | | | |— accounts.yml
| | | | |— audit.yml
| | | | |— kernel.yml
| | | | |— apparmor.yml
| | | | |— handlers/
| | | | |   |— main.yml
| | | | |— defaults/
| | | | |   |— main.yml
| | | | |— network_audit/
| | | | |— reporting/
| | | | |— scripts/
| | | | |   |— parse_lynis.py
| | | | |   |— compare_results.py
| | | | |   |— build_report.py
| | | | |— reports/
| | | | |   |— before/
| | | | |   |— after/
| | | | |   |— final/
| | | | |— README.md
```

Prérequis

Logiciels à installer

- Docker + Docker Compose
- Python 3.11+
- Ansible (`pip install ansible ansible-lint`)
- Git
- Lynis (sur le serveur cible ou dans l'image)

Compétences mobilisées

- Administration Linux (SSH, services, pare-feu)
 - YAML / Ansible (playbooks, rôles, variables)
 - Python (parsing de fichiers, génération de rapports)
 - Réseau TCP/IP de base
 - Lecture de configurations système et réseau
-

Jour 1 — Déploiement du lab et audit initial

1.1 Scénario initial volontairement vulnérable

Serveur Linux

Configurer volontairement les éléments suivants pour créer une baseline faible mais réaliste :

- `PermitRootLogin yes`
- `PasswordAuthentication yes`
- Pas de pare-feu actif
- Services inutiles démarrés (telnet, vsftpd, cups)
- `auditd` absent
- Journalisation minimale
- AppArmor inactif ou non configuré
- Mises à jour non appliquées

Équipements réseau simulés

- Telnet activé
- SNMP v1 ou v2c avec communauté "public"
- HTTP d'administration activé
- ACL de management absente
- Compte admin avec mot de passe par défaut

1.2 docker-compose.yml

```
services:
  linux-target:
    image: ubuntu:22.04
    container_name: linux-target
    ports:
      - "2222:22"
    networks:
      - audit_lab_net
    command: /bin/bash -c "apt-get update -qq &&
      apt-get install -y -qq openssh-server telnet vsftpd &&
      echo 'PermitRootLogin yes' >> /etc/ssh/sshd_config &&
      echo 'PasswordAuthentication yes' >> /etc/ssh/sshd_config &&
      service ssh start && tail -f /dev/null"
```

```

ansible-controller:
  image: python:3.11-slim
  container_name: ansible-controller
  volumes:
    - ./:/workspace
  working_dir: /workspace
  networks:
    - audit_lab_net
  command: /bin/bash -c "pip install -q ansible &&
    tail -f /dev/null"

networks:
  audit_lab_net:
    driver: bridge

```

1.3 Inventaire Ansible

```

# inventory/hosts.yml
all:
  children:
    linux:
      hosts:
        linux-target:
          ansible_host: 127.0.0.1
          ansible_port: 2222
          ansible_user: root
          ansible_password: "rootpass"
    network:
      hosts:
        router1:
          ansible_host: 10.10.0.21
          ansible_network_os: ios
          ansible_user: admin
          ansible_password: "admin"

```

1.4 Audit initial Linux

```

# playbooks/audit_linux.yml
- name: Audit Linux initial
  hosts: linux
  gather_facts: true
  tasks:
    - name: Get OS version
      command: cat /etc/os-release
      register: os_version

```

```
changed_when: false

- name: List active services
  command: systemctl list-units --type=service --state=running
  register: active_services
  changed_when: false

- name: Check listening ports
  command: ss -tlnp
  register: open_ports
  changed_when: false

- name: Check firewall status
  command: ufw status
  register: firewall_status
  changed_when: false
  ignore_errors: true

- name: Check SSH config
  command: grep -E "PermitRootLogin|PasswordAuthentication|MaxAuthTries" /etc/ssh/sshd_config
  register: ssh_config
  changed_when: false

- name: Check auditd presence
  command: systemctl is-active auditd
  register: auditd_status
  changed_when: false
  ignore_errors: true

- name: Run Lynis audit
  command: lynis audit system --quick --no-colors
  register: lynis_result
  changed_when: false
  ignore_errors: true

- name: Save audit results
  copy:
    content: "{{ lynis_result.stdout }}"
    dest: /tmp/lynis_before.txt

- name: Fetch Lynis report
  fetch:
    src: /tmp/lynis_before.txt
    dest: reports/before/lynis_before.txt
    flat: true
```

1.5 Audit initial réseau

```
# playbooks/audit_network.yml
- name: Audit réseau initial
  hosts: network
  gather_facts: false
  tasks:

    - name: Show version
      ios_command:
        commands: show version
      register: version_output

    - name: Show running config
      ios_command:
        commands: show running-config
      register: running_config

    - name: Check SNMP
      ios_command:
        commands: show snmp
      register: snmp_output

    - name: Save results
      local_action:
        module: copy
        content: |
          {
            "device": "{{ inventory_hostname }}",
            "version": "{{ version_output.stdout[0] }}",
            "snmp": "{{ snmp_output.stdout[0] }}"
          }
        dest: "reports/before/network_{{ inventory_hostname }}.json"
```

Jour 2 — Hardening automatisé

2.1 Variables du rôle linux_hardening

```
# roles/linux_hardening/defaults/main.yml
ssh_permit_root_login: "no"
ssh_password_authentication: "no"
ssh_max_auth_tries: 3
ssh_x11_forwarding: "no"
allowed_ssh_users:
  - ansible
```

```

linux_allowed_ports:
  - "22"

unused_services:
  - telnet
  - vsftpd
  - cups
  - xinetd

enable_auditd: true
enable_apparmor: true

audit_rules:
  - "-w /etc/passwd -p wa -k passwd_changes"
  - "-w /etc/shadow -p wa -k shadow_changes"
  - "-w /etc/ssh/sshd_config -p wa -k ssh_config"
  - "-w /var/log/auth.log -p wa -k auth_log"

```

2.2 Durcissement SSH

```

# roles/linux_hardening/tasks/ssh.yml
- name: Disable root SSH login
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
    line: "PermitRootLogin {{ ssh_permit_root_login }}"
  notify: Restart ssh

- name: Disable password authentication
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PasswordAuthentication'
    line: "PasswordAuthentication {{ ssh_password_authentication }}"
  notify: Restart ssh

- name: Reduce MaxAuthTries
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?MaxAuthTries'
    line: "MaxAuthTries {{ ssh_max_auth_tries }}"
  notify: Restart ssh

- name: Disable X11 forwarding
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?X11Forwarding'

```

```
    line: "X11Forwarding {{ ssh_x11_forwarding }}"
  notify: Restart ssh
```

2.3 Pare-feu

```
# roles/linux_hardening/tasks/firewall.yml
- name: Install ufw
  apt:
    name: ufw
    state: present
    update_cache: true

- name: Set default deny incoming
  ufw:
    default: deny
    direction: incoming

- name: Set default allow outgoing
  ufw:
    default: allow
    direction: outgoing

- name: Allow required ports
  ufw:
    rule: allow
    port: "{{ item }}"
    proto: tcp
  loop: "{{ linux_allowed_ports }}"

- name: Enable ufw
  ufw:
    state: enabled
```

2.4 Services inutiles

```
# roles/linux_hardening/tasks/services.yml
- name: Stop and disable unused services
  systemd:
    name: "{{ item }}"
    state: stopped
    enabled: false
  loop: "{{ unused_services }}"
  ignore_errors: true

- name: Remove telnet package
  apt:
```

```
name: telnet
state: absent
```

2.5 Journalisation avec auditd

```
# roles/linux_hardening/tasks/audit.yml
- name: Install auditd
  apt:
    name: auditd
    state: present

- name: Start and enable auditd
  systemd:
    name: auditd
    state: started
    enabled: true

- name: Add audit rules
  lineinfile:
    path: /etc/audit/rules.d/hardening.rules
    line: "{{ item }}"
    create: true
  loop: "{{ audit_rules }}"
  notify: Reload audit rules
```

2.6 Handlers

```
# roles/linux_hardening/handlers/main.yml
- name: Restart ssh
  systemd:
    name: ssh
    state: restarted

- name: Reload audit rules
  command: augenrules --load
```

2.7 Playbook hardening principal

```
# playbooks/harden_linux.yml
- name: Harden Linux server
  hosts: linux
  become: true
  roles:
    - role: linux_hardening
```

Jour 3 — Validation et rapport

3.1 Playbook de validation

```
# playbooks/validate.yml
- name: Validate hardening
  hosts: linux
  gather_facts: false
  tasks:

    - name: Run Lynis post-hardening
      command: lynis audit system --quick --no-colors
      register: lynis_after
      changed_when: false
      ignore_errors: true

    - name: Save post-hardening report
      copy:
        content: "{{ lynis_after.stdout }}"
        dest: /tmp/lynis_after.txt

    - name: Fetch post-hardening report
      fetch:
        src: /tmp/lynis_after.txt
        dest: reports/after/lynis_after.txt
        flat: true
```

3.2 Script Python de comparaison

```
# scripts/compare_results.py
import re
import json

def extract_lynis_score(filepath):
    with open(filepath, "r") as f:
        content = f.read()
    match = re.search(r"Hardening index\s*:\s*(\d+)", content)
    return int(match.group(1)) if match else None

def extract_warnings(filepath):
    warnings = []
    with open(filepath, "r") as f:
        for line in f:
            if "Warning" in line or "WARN" in line:
                warnings.append(line.strip())
    return warnings
```

```

def compare_reports(before_path, after_path):
    score_before = extract_lynis_score(before_path)
    score_after = extract_lynis_score(after_path)
    warnings_before = extract_warnings(before_path)
    warnings_after = extract_warnings(after_path)

    fixed = [w for w in warnings_before if w not in warnings_after]
    remaining = [w for w in warnings_after]

    result = {
        "score_before": score_before,
        "score_after": score_after,
        "improvement": score_after - score_before if score_before and score_after,
        "warnings_before": len(warnings_before),
        "warnings_after": len(warnings_after),
        "fixed_warnings": fixed,
        "remaining_warnings": remaining
    }
    return result

if __name__ == "__main__":
    result = compare_reports(
        "reports/before/lynis_before.txt",
        "reports/after/lynis_after.txt"
    )
    with open("reports/final/comparison.json", "w") as f:
        json.dump(result, f, indent=2, ensure_ascii=False)
    print(json.dumps(result, indent=2, ensure_ascii=False))

```

3.3 Script Python de génération du rapport HTML

```

# scripts/build_report.py
import json
from jinja2 import Template

with open("reports/final/comparison.json") as f:
    data = json.load(f)

template_str = """
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Rapport Audit & Hardening</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; color: #222; }
        h1 { color: #1a237e; }
        h2 { color: #283593; border-bottom: 2px solid #e8eaf6; padding-bottom: 5px; }
    </style>
</head>
<body>
    <h1>Rapport Audit & Hardening</h1>
    <h2>Résumé</h2>
    <table>
        <tr>
            <th>Statut</th>
            <th>Nombre</th>
        </tr>
        <tr>
            <td>Améliorations</td>
            <td>{{ data.fixed_warnings|length }}</td>
        </tr>
        <tr>
            <td>Problèmes persistants</td>
            <td>{{ data.remaining_warnings|length }}</td>
        </tr>
    </table>
</body>
</html>
"""
template = Template(template_str)

```

```

        .score { font-size: 2em; font-weight: bold; }
        .good { color: #2e7d32; }
        .bad { color: #c62828; }
        .warn { color: #f57f17; }
        table { border-collapse: collapse; width: 100%; margin-top: 20px; }
        th, td { border: 1px solid #bbb; padding: 8px 12px; text-align: left; }
        th { background-color: #e8eaf6; }
        ul { line-height: 1.8; }
    </style>
</head>
<body>

<h1>Rapport Audit & Hardening Lab</h1>

<h2>Scores Lynis</h2>
<table>
    <tr><th>Mesure</th><th>Valeur</th></tr>
    <tr><td>Score avant hardening</td>
        <td class="bad score">{{ data.score_before }} / 100</td></tr>
    <tr><td>Score après hardening</td>
        <td class="good score">{{ data.score_after }} / 100</td></tr>
    <tr><td>Amélioration</td>
        <td class="good"><strong>+{{ data.improvement }} points</strong></td></tr>
    <tr><td>Warnings avant</td><td class="warn">{{ data.warnings_before }}</td></tr>
    <tr><td>Warnings après</td><td>{{ data.warnings_after }}</td></tr>
</table>

<h2>Problèmes corrigés ({{ data.fixed_warnings | length }})</h2>
<ul>
{% for item in data.fixed_warnings %}
    <li class="good">{{ item }}</li>
{% endfor %}
</ul>

<h2>Warnings restants ({{ data.remaining_warnings | length }})</h2>
<ul>
{% for item in data.remaining_warnings %}
    <li class="warn">{{ item }}</li>
{% endfor %}
</ul>

</body>
</html>
"""

template = Template(template_str)
output = template.render(data=data)

with open("reports/final/report.html", "w") as f:
    f.write(output)

```

```
print("Rapport HTML généré : reports/final/report.html")
```

3.4 Pipeline GitLab CI

```
# .gitlab-ci.yml
stages:
  - lint
  - audit
  - report

lint-yaml:
  stage: lint
  image: python:3.11-slim
  script:
    - pip install -q ansible-lint yamllint
    - yamllint .
    - ansible-lint playbooks/

run-audit:
  stage: audit
  image: python:3.11-slim
  script:
    - pip install -q ansible
    - ansible-playbook playbooks/audit_linux.yml
  artifacts:
    paths:
      - reports/before/
    expire_in: 7 days

generate-report:
  stage: report
  image: python:3.11-slim
  script:
    - pip install -q jinja2
    - python scripts/compare_results.py
    - python scripts/build_report.py
  artifacts:
    paths:
      - reports/final/
    expire_in: 30 days
```

Politique de sécurité appliquée

Règles Linux

Règle	Sévérité	Référentiel
Root SSH interdit	Critique	ANSSI-BP-028
Auth par mot de passe désactivée	Critique	ANSSI-BP-028
Pare-feu actif avec politique deny-all	Critique	ANSSI-BP-028
Services inutiles désactivés	Élevé	ANSSI-BP-028
auditd actif	Élevé	ANSSI-BP-028
AppArmor actif	Moyen	ANSSI-BP-028
Mises à jour appliquées	Élevé	ANSSI

Règles réseau

Règle	Sévérité
Telnet interdit	Critique
HTTP admin désactivé	Élevé
SNMP v1/v2c signalé	Élevé
ACL VTY obligatoire	Élevé
Comptes par défaut supprimés	Critique

Planning 3 jours

Jour	Tâches	Livrables
Jour 1	docker-compose, inventaire Ansible, audit Linux initial, audit réseau initial	reports/ before/
Jour 2	Rôle linux_hardening (SSH, firewall, services, auditd, AppArmor), remédiation réseau légère	Serveur durci
Jour 3	Playbook validation, scripts Python, rapport HTML, README, CI facultatif	reports/ final/ , README.md

Exemple de README.md

```
# Audit & Hardening Lab

Automated security audit and Linux hardening lab using Ansible and Docker.

## Objectif

Démonstration d'une démarche complète de sécurisation :
audit initial → remédiation automatisée → validation.

## Architecture

- 1 serveur Ubuntu volontairement mal configuré
- 1 équipement réseau simulé (FakeNOS)
- 1 contrôleur Ansible

## Lancement rapide

git clone <repo>
cd audit-hardening-lab
docker-compose up -d
ansible-playbook playbooks/audit_linux.yml
ansible-playbook playbooks/harden_linux.yml
ansible-playbook playbooks/validate.yml
python scripts/compare_results.py
python scripts/build_report.py
```

Résultats

Mesure	Avant	Après
---	---	---
Score Lynis	~42/100	~78/100
Warnings critiques	6	1
Root SSH	Autorisé	Interdit
Pare-feu	Inactif	Actif (deny-all)
auditd	Absent	Actif

Référentiel

Basé sur ANSSI-BP-028 – Guide de recommandations de configuration d'un système C

Limites

- Lab simplifié (simulation réseau partielle)
- Pas de SIEM ni de PKI
- Conformité ANSSI partielle

Limites à assumer

Une présentation honnête des limites renforce la crédibilité technique du projet. Il est sain d'indiquer :

- Simulation réseau avec FakeNOS, pas d'équipements physiques réels.
- Conformité ANSSI partielle, non exhaustive.
- Pas de SIEM ni de PKI avancée.
- Pas de gestion multi-utilisateurs complète.
- Rapport HTML non certifié.

Extensions possibles

- Intégration de **fail2ban** pour la protection contre le brute-force SSH.
- Ajout de **OpenSCAP** pour une conformité CIS plus formelle.
- Tests **Molecule** pour valider les rôles Ansible.
- Dashboard de supervision avec **Grafana + Loki**.

- Gestion des secrets avec **Ansible Vault**.
- Inventaire multi-hôtes avec **Netbox**.

Ce que voit le jury MS-SIS

Compétence démontrée	Signal envoyé
Référentiel ANSSI-BP-028	Connaissance des standards officiels français
Lynis + score delta	Approche métrique, pas juste "j'ai durci"
Ansible idempotent	Infrastructure-as-Code appliqué à la sécurité
auditd + logging	Mindset SOC / traçabilité
Pipeline CI	Sécurité DevSecOps, pas ponctuelle
Limites documentées	Maturité technique et honnêteté

Rapport réalisé dans le cadre de la préparation d'une candidature au MS-SIS ESIEA.